

AgriCam

Cahier des charges



Diagnostic phytosanitaire explicable des cultures maraicheres,
fonctionnant sans reseau



Version 1.0 · 25 juillet 2026
Tomate · Piment · Oignon · 9 etats sanitaires
Memoire de fin de cycle · Bachelor IA & Big Data

1. Contexte et problème

Le maraîchage camerounais repose sur de petites exploitations sans encadrement technique systématique. Le diagnostic des maladies s'y fait à l'œil nu, tardivement, et les symptômes précoces de plusieurs atteintes se ressemblent au point qu'un producteur non spécialisé confond couramment une anthracnose débutante avec une tache bactérienne — deux maladies dont les traitements n'ont rien à voir.

Deux conséquences directes. Des pertes de récolte évitables, d'abord. Des traitements inutiles ensuite : le réflexe le plus fréquent face au doute est d'appliquer un fongicide, qui reste sans effet sur une bactériose, sur une virose et sur un trouble physiologique — soit, dans le référentiel d'AgriCam, quatre classes sur neuf.

Les outils numériques existants visent de grandes exploitations équipées, ou supposent une connexion permanente que les zones de production n'ont pas.

Ce qui distingue AgriCam

Le verrou n'est pas la reconnaissance, c'est la localisation. Un classifieur entraîné sur des fruits cadrés ne sait pas où regarder dans une photo de terrain — une branche, du feuillage, du sol. AgriCam résout ce problème en amont, par un détecteur qui isole chaque fruit avant classification. Cette conclusion est le fil rouge scientifique du mémoire : les classes les plus faibles en classification sont simultanément les plus dispersées sous Grad-CAM et les moins bien localisées par le détecteur — trois méthodes indépendantes, un seul diagnostic.

2. Objectifs

Objectif	Indicateur de réussite
O1 Diagnostiquer l'état sanitaire d'un fruit à partir d'une photo	Exactitude ≥ 90 % sur le jeu de test
O2 Fonctionner intégralement sans réseau	Diagnostic complet en mode avion, après premier chargement
O3 Rendre la décision lisible par un non-spécialiste	Carte d'activation superposée + verdict en langage courant
O4 Transformer le diagnostic en action	Conduite à tenir affichée pour chacune des 9 classes
O5 Tenir sur un téléphone d'entrée de gamme	< 40 Mo à télécharger, < 3 s par diagnostic
O6 Constituer un jeu de données camerounais	Signalements utilisateurs horodatés et géolocalisés

3. Périmètre

Inclus. Reconnaissance de 9 états sanitaires sur l'organe récolté — fruit ou bulbe. Détection préalable sur tomate. Explicabilité visuelle. Conduite à tenir. Fonctionnement hors ligne. Historique et suivi de parcelle. Cartographie épidémiologique.

Exclu à ce stade. Maladies foliaires et racinaires. Prescription de produits phytosanitaires avec posologie — l'application oriente vers un technicien pour ce choix, et l'assume. Diagnostic vétérinaire ou sur d'autres cultures. Détection préalable sur piment et oignon, faute de jeu annoté en boîtes.

Hypothèse d'usage. Une photo, un organe. Les photos de parcelle entière relèvent du balayage vidéo (EF-22), pas du diagnostic unitaire.

4. Acteurs

Acteur	Besoin	Contrainte
Producteur	Savoir ce qu'a son fruit et quoi faire	Souvent peu lettré, téléphone modeste, pas de réseau au champ
Technicien d'appui	Suivre plusieurs producteurs, agréger	Se déplace, a besoin d'exports
Coopérative / IRAD	Voir les foyers, anticiper	Besoin de données agrégées, pas de photos individuelles

5. Référentiel des classes

#	Classe	Culture	Nature	Gravité
0	Bulbe atteint	Oignon	Multiple	Atteint
1	Oignon sain	Oignon	—	Sain
2	Anthraxnose du piment	Piment	Champignon	Atteint
3	Piment sain	Piment	—	Sain
4	Anthraxnose de la tomate	Tomate	Champignon	Atteint
5	Tache bactérienne	Tomate	Bactérie	Atteint
6	Pourriture apicale	Tomate	Physiologique	À surveiller
7	Tomate saine	Tomate	—	Sain
8	Virus de la maladie bronzée	Tomate	Virus	Grave

L'ordre est **contractuel**. Il indexe les sorties du softmax. Une permutation rendrait tous les diagnostics faux sans lever la moindre erreur. Un contrôle d'intégrité compare le référentiel de l'application au fichier livré avec les poids et refuse de démarrer en cas de divergence.

6. Exigences fonctionnelles

Priorité : **O** obligatoire · **S** souhaitable · **F** facultatif Statut :

Fait

 fait ·

En cours

 en cours ·

A faire

 à faire

6.1 Acquisition et contrôle qualité

Réf.	Exigence	Prio	Statut
EF-01	Prise de photo par l'appareil ou choix dans la galerie	O	Fait
EF-02	Viseur avec consignes de cadrage explicites	O	Fait
EF-03	Contrôle de netteté avant inférence, par variance du laplacien	O	Fait
EF-04	Contrôle d'exposition et de contraste	O	Fait
EF-05	Message de rejet nommant le défaut et le geste correctif	O	Fait
EF-06	Possibilité de passer outre le rejet	S	Fait
EF-07	Géolocalisation facultative, jamais bloquante	S	Fait

Ce contrôle amont répond directement à un constat du mémoire : les échecs de localisation Grad-CAM se concentrent sur les images floues ou mal éclairées, où l'attention du réseau glisse vers l'arrière-plan.

6.2 Pipeline de diagnostic

Réf.	Exigence	Prio	Statut
EF-08	Détection des fruits par YOLOv8n dans le navigateur	O	Fait
EF-09	Recadrage automatique de chaque fruit détecté	O	Fait
EF-10	Classification par EfficientNetB3 sur chaque vignette	O	Fait
EF-11	Repli en diagnostic pleine image si aucune détection, signalé	O	Fait
EF-12	Diagnostic multi-fruits sur une seule photo	O	Fait
EF-13	Taux d'infestation calculé sur les fruits détectés	O	Fait
EF-14	Rejet informé sous le seuil de confiance	O	Fait
EF-15	Filtre hors-domaine par distance de Mahalanobis	S	A faire

6.3 Explicabilité

Réf.	Exigence	Prio	Statut
EF-16	Carte d'activation superposée à la vignette	O	Fait
EF-17	Bascule montrer / masquer les zones analysées	O	Fait
EF-18	Bande de sévérité affichant la distribution des probabilités	O	Fait
EF-19	Photo annotée des boîtes, colorées par gravité, sélectionnables	O	Fait
EF-20	Avertissement quand les zones chaudes ne couvrent pas la lésion	S	A faire

Point technique remarquable. TensorFlow.js ne calcule pas de gradients sur un modèle converti : Grad-CAM y est inaccessible. Mais l'architecture s'achevant par `GlobalAveragePooling` → `Dense`, les coefficients de Grad-CAM se réduisent aux poids de la couche dense — $\alpha_k = w_{kc}/Z$. La carte se calcule donc par simple combinaison pondérée, mathématiquement identique à Grad-CAM, sans aucun gradient.

6.4 Conduite à tenir

Réf.	Exigence	Prio	Statut
EF-21	Fiche d'action pour chacune des 9 classes	O	Fait
EF-22	Niveau d'urgence explicite	O	Fait
EF-23	Gestes ordonnés, formulés en actions concrètes	O	Fait
EF-24	Encadré « à ne pas faire »	O	Fait
EF-25	Conseils de prévention, repliés par défaut	S	Fait
EF-26	Calendrier de rappels après diagnostic	F	A faire

6.5 Historique et suivi

Réf.	Exigence	Prio	Statut
EF-27	Conservation locale de toutes les consultations	O	Fait
EF-28	Écran d'historique : liste, détail, suppression	O	A faire
EF-29	Rattachement d'une consultation à une parcelle	S	Fait
EF-30	Indice de santé de parcelle sur fenêtre glissante	S	Fait
EF-31	Suivi temporel d'un même plant, avec comparaison	S	A faire
EF-32	Export PDF d'un diagnostic	S	A faire

6.6 Dimension collective

Réf.	Exigence	Prio	Statut
EF-33	Carte épidémiologique des diagnostics géolocalisés	S	A faire
EF-34	Synchronisation différée, métadonnées uniquement	S	En cours
EF-35	Alerte de voisinage sur foyer détecté	S	A faire
EF-36	Signalement d'un diagnostic douteux, discret	O	Fait
EF-37	Espace d'échange entre producteurs	F	A faire

6.7 Accompagnement

Réf.	Exigence	Prio	Statut
EF-38	Assistant conversationnel adossé à la documentation IRAD	S	A faire
EF-39	Lecture vocale des conduites à tenir	S	A faire
EF-40	Interface français / anglais	S	A faire
EF-41	Mode technicien : plusieurs producteurs, export CSV	F	A faire

7. Fonctionnalités distinctives

Ces cinq points n'existent dans aucun projet comparable identifié. Ce sont eux qui portent la démonstration en soutenance.

FD-01 — Double lecture croisée ★

Le détecteur YOLO ne se contente pas de localiser : il classe aussi, sur cinq états de la tomate. Le classifieur EfficientNetB3 classe indépendamment sur neuf. **Deux réseaux d'architectures différentes, entraînés séparément, donnent leur avis sur le même fruit.**

Leur accord devient un indice de fiabilité gratuit. Concordance → confiance renforcée. Divergence → l'application le signale et invite à reprendre la photo. Aucune ligne de calcul supplémentaire, aucun modèle à entraîner : l'information était déjà là, inexploitée.

Priorité : haute. Effort : faible. Effet en soutenance : très fort.

FD-02 — Estimation de la perte évitée ★

À partir du taux d'infestation, du nombre de pieds déclarés et du prix courant au kilo, l'application chiffre en FCFA la récolte menacée, et ce qu'un traitement à temps permettrait de sauver.

C'est ce qui transforme un pourcentage abstrait en argument : « 33 % d'infestation sur 200 pieds, soit environ 45 000 FCFA de récolte en jeu. » Un producteur comprend immédiatement. Un jury aussi.

Priorité : haute. Effort : faible.

FD-03 — Balayage de rang

L'utilisateur filme lentement un rang. L'application analyse une image sur dix, cumule les détections, et produit un indice d'infestation de parcelle plutôt qu'un verdict sur un fruit isolé.

C'est le passage du diagnostic unitaire à l'échelle qui intéresse réellement un producteur : sa parcelle, pas sa tomate.

Priorité : moyenne. Effort : élevé.

FD-04 — Évolution d'une lésion dans le temps

Photographier le même fruit à sept jours d'intervalle. L'application mesure la variation de surface atteinte et dit si l'atteinte progresse, stagne ou régresse — donc si le traitement appliqué fonctionne.

Aucun outil comparable ne le propose. C'est pourtant la question que se pose tout producteur après avoir traité.

Priorité : moyenne. Effort : moyen.

FD-05 — Démonstration hors ligne

Couper le Wi-Fi devant le jury, activer le mode avion, et faire un diagnostic complet — détection, classification, carte de chaleur, conduite à tenir. Sans serveur, sans latence, sans données envoyées.

Ce n'est pas une fonctionnalité mais un **geste de démonstration**, et c'est probablement le moment le plus convaincant de la soutenance. Il faut le préparer et le répéter.

Priorité : haute. Effort : nul — c'est déjà le cas.

8. Exigences non fonctionnelles

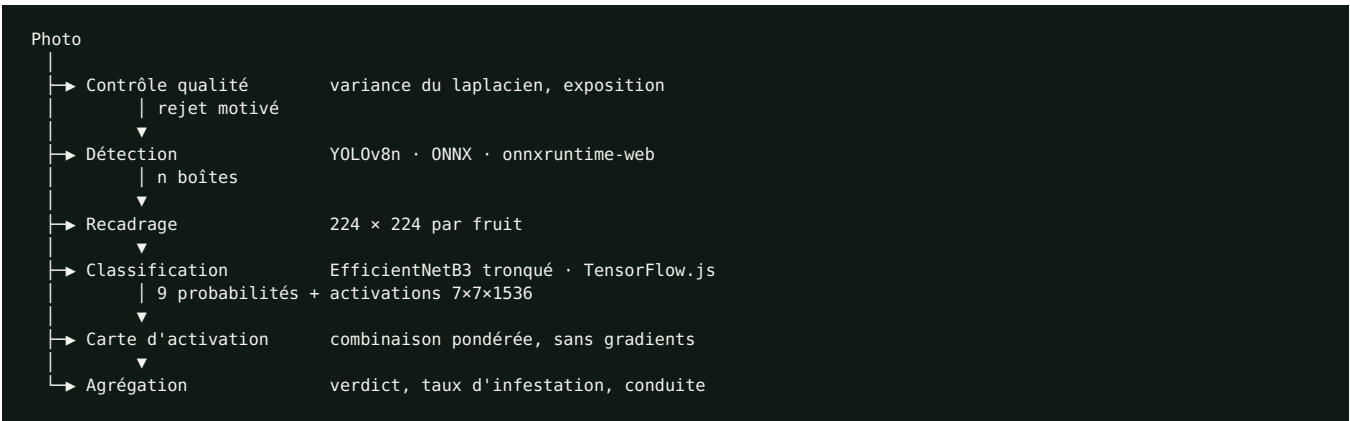
Réf.	Exigence	Cible	Mesuré
ENF-01	Exactitude sur le jeu de test	$\geq 90 \%$	95,6 %
ENF-02	Temps d'un diagnostic complet	$< 3 \text{ s}$	647 ms
ENF-03	Poids total à télécharger, une seule fois	$< 40 \text{ Mo}$	35,4 Mo
ENF-04	Fonctionnement sans réseau après premier chargement	Total	Fait
ENF-05	La photo ne quitte jamais l'appareil	Absolu	Fait
ENF-06	Lisibilité en plein soleil	Contraste $\geq 7:1$	Fait
ENF-07	Zone tactile minimale	48 px	Fait
ENF-08	Largeur d'écran minimale	320 px	Fait
ENF-09	Navigation au clavier, focus visible	Complète	Fait
ENF-10	Respect de <code>prefers-reduced-motion</code>	Oui	Fait
ENF-11	Tout message d'erreur dit quoi faire	Systématique	Fait

Composition des 35,4 Mo. Les deux réseaux pèsent 22,5 Mo, le moteur d'inférence WebAssembly 10,1 Mo, l'application elle-même 2,8 Mo. La distribution complète d'onnxruntime en pèse 88 : n'est embarquée que la variante SIMD, les déclinaisons multithread, WebGPU et entraînement étant sans usage ici.

Vie privée

L'image reste sur le téléphone. Seules des métadonnées — classe, horodatage, position approximative — remontent, et uniquement sur action explicite. Le signalement d'un diagnostic douteux est le seul cas où une image peut être transmise, avec accord.

9. Architecture



Couche	Technologie	Justification
Interface	React + TypeScript + Vite	Typage strict, rechargement instantané
Détection	onnxruntime-web	Conversion directe depuis Ultralytics, fiable
Classification	TensorFlow.js, GraphModel quantifié uint8	Poids divisé par quatre
Stockage	IndexedDB	Volume, structure, disponible hors ligne
Hors ligne	Service worker, precache	Les deux réseaux mis en cache
Cartographie	MapLibre GL	Libre, fonds téléchargeables

Pourquoi deux moteurs d'inférence. La conversion de poids Ultralytics vers ONNX est directe et fiable ; vers TensorFlow.js elle ne l'est pas. Le coût — deux bibliothèques — est assumé au bénéfice de la robustesse.

10. Points de vigilance techniques

Quatre pièges rencontrés pendant le développement, tous **silencieux** : aucun ne lève d'erreur, tous produisent un résultat plausible et faux.

Piège	Symptôme	Parade
Double normalisation	Précision effondrée, sans erreur	EfficientNet normalise en interne : pixels transmis bruts, sur [0, 255]
Constantes de normalisation figées	95,6 % → 80,7 %	<code>finalize_state()</code> après chargement des poids
<code>load_weights(by_name=True)</code>	Réseau resté aléatoire, silencieux	Chargement par nom explicite, avec décompte des tenseurs
Ordre des classes	Diagnostics faux, confiance normale	Contrôle d'intégrité au démarrage

Ce que cela impose. La seule vérification qui tranche est empirique : faire prédire de vraies images et comparer à une exactitude de référence connue. C'est une exigence du projet, pas une précaution optionnelle.

11. Livrables

Livrable	Forme	État
Application web progressive	Code source + build	En cours
Modèles convertis	TF.js + ONNX, 22,5 Mo	Fait
Moteur d'inférence embarqué	onnxruntime SIMD, 10,1 Mo	Fait
Chaîne d'export reproductible	Scripts Python documentés	Fait
Scripts de vérification	Contrôle empirique des poids	Fait
Mémoire	Document Word, normes KEYCE	En cours
Support de soutien	Diaporama + démonstration	A faire

12. Trajectoire

Étape 1 — rendre l'existant démontrable. Écran d'historique, export PDF, double lecture croisée (FD-01), estimation économique (FD-02), correction des bandes noires sur les vignettes de bord.

Étape 2 — dimension collective. Carte épidémiologique, synchronisation différée, alerte de voisinage, suivi temporel.

Étape 3 — accompagnement. Assistant conversationnel, lecture vocale, bilingue, mode technicien.

Annoncé en perspectives, non réalisé. Langues locales à l'oral, application native, capteurs de terrain, prédiction d'épidémie, extension aux maladies foliaires.

13. Critères d'acceptation

L'application est considérée comme livrable lorsque, **simultanément** :

- Un diagnostic complet aboutit en mode avion sur un téléphone d'entrée de gamme.
- L'exactitude mesurée sur le jeu de test dépasse 90 %.
- Les neuf classes disposent d'une conduite à tenir vérifiée.
- Une photo floue est rejetée avec un message actionnable.

5. Une prédiction sous le seuil affiche l'incertitude au lieu de trancher.
 6. Une photo multi-fruits produit un taux d'infestation cohérent.
 7. La carte d'activation se superpose visiblement à la lésion.
 8. Le contrôle d'intégrité du référentiel de classes passe au démarrage.
 9. Aucune zone tactile ne descend sous 48 px, aucun texte sous 14 px.
 10. La démonstration hors ligne a été répétée intégralement.
-

14. Limites assumées

Elles doivent être énoncées franchement en soutenance ; les masquer serait le plus sûr moyen d'y être ramené par une question.

Le détecteur ne couvre que la tomate. Seule source annotée en boîtes disponible. Piment et oignon basculent en diagnostic pleine image, ce que l'interface signale.

Aucune validation terrain camerounaise. Le corpus provient de sources étrangères. Les performances sont mesurées sur ce corpus, non sur des cultures locales photographiées en conditions réelles. C'est la limite principale, et le mécanisme de signalement est précisément conçu pour commencer à la lever.

Le déséquilibre entre classes subsiste, traité par pondération et augmentation, non résolu.

La calibration n'a pas été évaluée. L'application affiche un pourcentage de confiance dont la justesse probabiliste n'est pas établie. L'affichage est plafonné à 99 % pour ne pas surexposer cette incertitude.

Neuf classes, pas toutes les maladies. Un fruit atteint d'autre chose sera rangé dans la classe la plus proche. Le filtre hors-domaine (EF-15) vise à limiter ce risque.